

Supervised ML: Regression & Classification

Course 1 — detailed study notes (Weeks 1–3)

Math + intuition + implementation

1. Week 1 — Foundations

1.1 What machine learning is

Arthur Samuel: a field that lets computers learn without being explicitly programmed. Tom Mitchell (formal): a program learns from experience E on task T measured by P if its performance at T , measured by P , improves with E .

Two main paradigms in this course:

- **Supervised** — you give the algorithm data with the “right answers” (x, y) and it learns the mapping $x \rightarrow y$. Splits into *regression* (continuous output, e.g. house price) and *classification* (discrete classes, e.g. benign/malignant).
- **Unsupervised** — only x , no labels. The algorithm finds structure: *clustering* (Google News grouping), anomaly detection, dimensionality reduction.

1.2 Notation (used everywhere)

m	number of training examples
x	input feature(s)
y	output / target
$(x^{(i)}, y^{(i)})$	the i -th training example
$\hat{y}$	the prediction (model’s estimate of y)
w, b	parameters (weight, bias) — this course uses w, b not θ

1.3 Linear regression with one variable

The model (hypothesis) is a straight line:

$$f_{w,b}(x) = wx + b = \hat{y}.$$

w is the slope, b the intercept. Training = finding w, b that make the line fit the data.

1.4 Cost function — squared error

Measures how wrong the line is. Average the squared gap between prediction and truth:

$$J(w, b) = \frac{1}{2m} \sum_{i=1}^m (f_{w,b}(x^{(i)}) - y^{(i)})^2 = \frac{1}{2m} \sum_{i=1}^m (\hat{y}^{(i)} - y^{(i)})^2.$$

- The $\frac{1}{2}$ is cosmetic — it cancels the 2 that appears when you differentiate.
- Squaring punishes big errors more and keeps everything positive.
- For linear regression J is a **convex** (bowl-shaped) surface, so it has exactly one minimum — no local minima to get stuck in.

Intuition: with b fixed, plotting J against w gives a parabola; the bottom is the best w . With both w, b free, J is a 3-D bowl and its contour plot is a set of ellipses — the centre is the optimum.

1.5 Gradient descent

An iterative way to reach the bottom of the bowl. Repeat until convergence, updating both parameters *simultaneously*:

$$w := w - \alpha \frac{\partial J}{\partial w}, \quad b := b - \alpha \frac{\partial J}{\partial b}.$$

α (learning rate) sets the step size. “Simultaneous” means compute both derivatives using the *old* values first, then assign.

Pitfall: updating w and then using the *new* w to update b is *not* gradient descent and behaves wrongly. Compute both temp values, then update.

The derivatives (worked out for linear regression)

Substituting $f = wx + b$ and differentiating J :

$$\begin{aligned} \frac{\partial J}{\partial w} &= \frac{1}{m} \sum_{i=1}^m (f_{w,b}(x^{(i)}) - y^{(i)}) x^{(i)}, \\ \frac{\partial J}{\partial b} &= \frac{1}{m} \sum_{i=1}^m (f_{w,b}(x^{(i)}) - y^{(i)}). \end{aligned}$$

Because J is convex here, gradient descent always converges to the global minimum (given a small enough α).

Choosing the learning rate α

- Too small $\Rightarrow$ convergence is painfully slow.
- Too large $\Rightarrow$ you overshoot; J can oscillate or diverge (increase).
- **Diagnostic:** plot J vs. iteration number (the “learning curve”). It should decrease every step and flatten out. If it ever rises, α is too big.
- Try a geometric ladder: 0.001, 0.003, 0.01, 0.03, 0.1, 0.3, ...
- Near the minimum the gradient shrinks, so steps shrink automatically — no need to reduce α by hand.

This is **batch** gradient descent: every step uses *all* m examples.

2. Week 2 — Multiple features

2.1 Model with n features

Now each example has features $x_1, \dots, x_n$:

$$f_{\mathbf{w},b}(\mathbf{x}) = w_1x_1 + w_2x_2 + \dots + w_nx_n + b = \mathbf{w} \cdot \mathbf{x} + b,$$

where $\mathbf{w} = (w_1, \dots, w_n)$ and $\mathbf{x} = (x_1, \dots, x_n)$ and $\cdot$ is the dot product. This is **multiple linear regression** (not “multivariate”).

2.2 Vectorization

Write it as a dot product and let NumPy do it in one fast call:

```
f = np.dot(w, x) + b
```

Vectorized code is shorter *and* much faster than a Python loop — the library runs the multiply-add in parallel. Use it everywhere.

2.3 Gradient descent for n features

Same shape as before, now one update per feature $j = 1 \dots n$:

$$w_j := w_j - \frac{\alpha}{m} \sum_{i=1}^m (f_{w,b}(\mathbf{x}^{(i)}) - y^{(i)}) x_j^{(i)},$$
$$b := b - \frac{\alpha}{m} \sum_{i=1}^m (f_{w,b}(\mathbf{x}^{(i)}) - y^{(i)}).$$

2.4 Feature scaling

When features span very different ranges (e.g. size 300–2000 vs. bedrooms 1–5), the cost contours become long thin ellipses and gradient descent zig-zags slowly. Rescale so all features are comparable ($\approx [-1, 1]$).

Three common methods:

$$\begin{aligned} \text{Max scaling: } x_j &\leftarrow \frac{x_j}{\max_j} \\ \text{Mean normalization: } x_j &\leftarrow \frac{x_j - \mu_j}{\max_j - \min_j} \\ \text{Z-score: } x_j &\leftarrow \frac{x_j - \mu_j}{\sigma_j} \end{aligned}$$

Rule of thumb: aim for roughly -1 to 1 ; ranges up to about $[-3, 3]$ or down to $[-\frac{1}{3}, \frac{1}{3}]$ are fine, beyond that rescale.

2.5 Checking convergence & feature engineering

- **Convergence test:** declare done when J decreases by less than a small ϵ (e.g. 10^{-3}) in one iteration — but reading the learning-curve plot is more reliable.
- **Feature engineering:** build better features from raw ones. E.g. from frontage and depth create `area = frontage*depth`, often a stronger predictor.

2.6 Polynomial regression

To fit curves, just add powers of x as new features and run ordinary linear regression:

$$f_{w,b}(x) = w_1x + w_2x^2 + w_3x^3 + b, \quad \text{or use } \sqrt{x} \text{ etc.}$$

With powers, feature scaling is *essential* — x^3 explodes in magnitude and dwarfs the linear term otherwise.

3. Week 3 — Classification

3.1 Why not linear regression for classification

y is discrete (0 or 1). A straight line can output values outside $[0, 1]$, and a single outlier can swing the fitted threshold badly. We need an output confined to $(0, 1)$.

3.2 Logistic regression & the sigmoid

$$f_{\mathbf{w},b}(\mathbf{x}) = g(\mathbf{w} \cdot \mathbf{x} + b), \quad g(z) = \frac{1}{1 + e^{-z}}.$$

The sigmoid g is S-shaped, passes through 0.5 at $z = 0$, and asymptotes to 0 and 1. The output is read as a probability:

$$f_{\mathbf{w},b}(\mathbf{x}) = P(y=1 \mid \mathbf{x}; \mathbf{w}, b).$$

So $f = 0.7$ means “70% chance $y = 1$.” Note $P(y=0) + P(y=1) = 1$.

3.3 Decision boundary

Predict $\hat{y} = 1$ when $f \geq 0.5$, i.e. when $z = \mathbf{w} \cdot \mathbf{x} + b \geq 0$; predict 0 otherwise. The boundary is the set $\{\mathbf{x} : \mathbf{w} \cdot \mathbf{x} + b = 0\}$ — a property of the parameters, not the data. Adding polynomial features (e.g. $x_1^2 + x_2^2$) makes it a circle or other non-linear shape.

Worked example: with $b = -3$, $w_1 = 1$, $w_2 = 1$, the boundary is $-3 + x_1 + x_2 = 0$, i.e. the line $x_1 + x_2 = 3$. Points above predict 1, below predict 0.

3.4 Cost function — logistic loss

Squared error on a sigmoid is non-convex (many local minima), so we use log loss:

$$L(f, y) = \begin{cases} -\log(f) & \text{if } y = 1 \\ -\log(1 - f) & \text{if } y = 0 \end{cases}$$

The intuition: if $y = 1$ and $f \rightarrow 0$, the loss $\rightarrow \infty$ (heavily punish a confident wrong answer); if $f \rightarrow 1$ the loss $\rightarrow 0$. Combine the two cases into one line:

$$J(\mathbf{w}, b) = -\frac{1}{m} \sum_{i=1}^m \left[y^{(i)} \log(f(\mathbf{x}^{(i)})) + (1 - y^{(i)}) \log(1 - f(\mathbf{x}^{(i)})) \right].$$

This cost is convex, so gradient descent finds the global minimum. (It comes from maximum-likelihood estimation.)

3.5 Gradient descent for logistic regression

Remarkably, the update is *algebraically identical* to linear regression — only the definition of f changed (sigmoid instead of line):

$$w_j := w_j - \frac{\alpha}{m} \sum_{i=1}^m (f(\mathbf{x}^{(i)}) - y^{(i)}) x_j^{(i)},$$
$$b := b - \frac{\alpha}{m} \sum_{i=1}^m (f(\mathbf{x}^{(i)}) - y^{(i)}).$$

4. Week 3 — Overfitting & Regularization

4.1 Under- vs over-fitting

	Underfit	Overfit
Also called	high bias	high variance
Symptom	too simple, misses trend	too complex, fits noise
On new data	poor	poor (memorized training)

Ways to address overfitting

(1) collect more training data; (2) use fewer features (feature selection); (3) **regularization** — keep all features but shrink the parameters.

4.2 Regularization idea

Add a penalty on large w_j to the cost, discouraging the model from relying too heavily on any feature. Convention: *penalize the w_j , not b* .

Regularized linear regression

$$J(\mathbf{w}, b) = \frac{1}{2m} \sum_{i=1}^m (f(\mathbf{x}^{(i)}) - y^{(i)})^2 + \frac{\lambda}{2m} \sum_{j=1}^n w_j^2.$$

Update (for each j):

$$w_j := w_j \left(1 - \alpha \frac{\lambda}{m}\right) - \frac{\alpha}{m} \sum_{i=1}^m (f(\mathbf{x}^{(i)}) - y^{(i)}) x_j^{(i)}.$$

The factor $(1 - \alpha\lambda/m) < 1$ shrinks w_j a little every step — “weight decay.” The b update is unchanged.

Regularized logistic regression

$$J(\mathbf{w}, b) = -\frac{1}{m} \sum_{i=1}^m \left[y^{(i)} \log f + (1 - y^{(i)}) \log(1 - f) \right] + \frac{\lambda}{2m} \sum_{j=1}^n w_j^2.$$

Gradient has the same added $\frac{\lambda}{m} w_j$ term as above.

Choosing λ : too large $\Rightarrow$ underfit (all $w_j \rightarrow 0$, model becomes flat line = b); too small $\Rightarrow$ overfit. Tune it on a validation set.

5. Implementation cheatsheet (labs)

NumPy core:

```
f = np.dot(w, x) + b          # vectorized prediction
cost = (1/(2*m)) * np.sum((f - y)**2)  # linear reg cost
g = 1 / (1 + np.exp(-z))      # sigmoid
```

scikit-learn — linear regression with scaling:

```
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import SGDRegressor, LinearRegression

scaler = StandardScaler()
X_norm = scaler.fit_transform(X)          # z-score scaling
model = SGDRegressor(max_iter=1000)      # gradient-descent based
```

```
model.fit(X_norm, y)
w, b = model.coef_, model.intercept_
```

scikit-learn — logistic regression:

```
from sklearn.linear_model import LogisticRegression
clf = LogisticRegression()
clf.fit(X, y)
clf.predict(X_new)           # class labels
clf.predict_proba(X_new)     # probabilities
```

The unifying idea of Course 1: linear and logistic regression share the *same* gradient-descent update $w_j := w_j - \frac{\alpha}{m} \sum_i (f(\mathbf{x}^{(i)}) - y^{(i)}) x_j^{(i)}$; only f differs (line vs. sigmoid). Regularization just adds $\frac{\lambda}{m} w_j$ to that gradient.